

QUESTION 1

Q1(a): Find the time complexity of the given functions. | Marks: 6

func1(int n) | Marks: 2

Model Answer:

The outer loop runs from 1 to n , contributing $O(n)$.

The middle loop also runs from 1 to n , contributing $O(n)$.

The inner loop runs from 1 to 10, which is constant, so $O(1)$.

Overall time complexity:

$$T(n) = O(n) \times O(n) \times O(1) = O(n^2).$$

func2(int n) | Marks: 2

Model Answer:

The outer loop runs from 0 to $2n$, giving $O(n)$.

The middle loop runs up to $i/3$, which in the worst case is $O(n)$.

The inner loop runs up to j^2 , which becomes $O(n^2)$ in the worst case.

Overall time complexity:

$$T(n) = O(n) \times O(n) \times O(n^2) = O(n^4).$$

func3(int n) | Marks: 2

Model Answer:

All three nested loops run up to n .

Overall time complexity:

$$T(n) = O(n) \times O(n) \times O(n) = O(n^3).$$

Q1(b): Optimized algorithm to find days until next greater rainfall. | Marks: 9

Model Answer:

This problem is solved using a **stack-based approach** in $O(n)$ time.

The algorithm traverses the rainfall array from right to left while maintaining a stack that stores indices of days with greater rainfall.

For each day:

- Pop elements from the stack while the current rainfall is greater or equal.
- If the stack is not empty, the difference between indices gives the number of days.
- Otherwise, the answer is 0.

This ensures each element is pushed and popped at most once.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

QUESTION 2

Q2(a): Evaluate the given postfix expression using stack. | Marks: 5

Model Answer:

The postfix expression is evaluated using a stack by scanning tokens from left to right.

Operands are pushed onto the stack.

When an operator is encountered, the top two elements are popped, the operation is applied, and the result is pushed back.

Final evaluated result: **95**

Q2(b): Perform the given queue operations. | Marks: 10

Part 1: Push at rear, Pop from front | Marks: 5

Model Answer:

Elements are inserted at the rear of Queue Q1 and removed from the front.

After each iteration, the front and rear pointers are updated correctly.

The intermediate states of Q1 and Q2 follow circular queue behavior.

Part 2: Push at front, Pop from rear | Marks: 5

Model Answer:

Elements are inserted at the front and removed from the rear of the queue.

The circular nature of the queue is preserved, and pointer updates ensure correct order after each iteration.

QUESTION 3

Q3(a): Merge two sorted linked lists. | Marks: 6

Model Answer:

The two linked lists are merged by comparing node values one by one.

A dummy node is used to simplify pointer handling.

At each step, the smaller node is linked to the result list, and the pointer is advanced.

After one list is exhausted, the remaining nodes of the other list are appended.

This approach maintains sorted order efficiently.

Q3(b): Insert an element into a sorted array using Binary Search. | Marks: 9

Model Answer:

Binary Search is used to find the correct insertion position in $O(\log n)$ time.

Once the position is found, elements are shifted to the right, and the new element is inserted.

Final array after insertion:

{10, 20, 25, 30, 40, 50}

Overall Time Complexity: **$O(n)$** (due to shifting).

QUESTION 4

Q4(a): Apply Selection Sort and state whether it is stable. | Marks: 5

Model Answer:

Selection Sort repeatedly selects the minimum element from the unsorted portion and swaps it with the first unsorted element.

After successive iterations, the array becomes sorted in ascending order.

Final sorted array: **[10, 12, 36, 45, 78]**

Selection Sort is **not stable** because swapping can change the relative order of equal elements.

Q4(b): Identify correct statements related to linked list manipulation. | Marks: 10

Model Answer:

Correct pointer comparisons, node traversal, and reassignment logic are required to correctly identify and reposition minimum and maximum nodes in a circular linked list.

Conditions involving curr, prev, maxNode, and minNode must ensure traversal does not break circularity and updates are performed safely.